

CSCE 641 COMPUTER GRAPHICS · MODULE 07 · LECTURE NOTES

The Programmable Pipeline: Shaders in Practice

Moving everything you just built onto the GPU.

MODULE OUTCOMES

Map each stage of your software rasterizer onto its GPU counterpart.

Write vertex and fragment shaders and explain the data flow between them.

Explain SIMD execution and why branching costs what it does.

Use uniforms, buffers, and textures with an understanding of their cost.

Capture and read a frame in RenderDoc.

1 The same pipeline, in hardware

Project 1 built every stage by hand. Moving to the GPU changes who executes each stage, not what the stages are. That is the whole pedagogical reason for doing the software rasterizer first: nothing below is new.

In your software rasterizer	On the GPU	Your control
The loop applying $P \cdot V \cdot M$ to each vertex	Vertex shader	Fully programmable.
Your near-plane clip and backface test	Fixed-function clip/cull	Configured through state, not written.
Your edge-function coverage loop	Rasterizer	None. Dedicated silicon, with a specified fill rule.
Your perspective-correct barycentric lerp	Interpolators	Controlled by qualifiers (smooth, flat, noperspective).
Your shade() function	Fragment shader	Fully programmable.
Your depth compare and write	ROP / output merger	Configured through depth and blend state.

2 Shader anatomy

A vertex shader runs once per vertex and must write `gl_Position` in clip space. Anything else it writes becomes a per-vertex output, which the rasterizer interpolates across the triangle — with perspective correction, exactly as derived in Module 04 — and delivers to the fragment shader.

```

// ---- vertex ----
#version 450 core
layout(location=0) in vec3 aPos;
layout(location=1) in vec3 aNormal;

uniform mat4 uMVP, uModel;
uniform mat3 uNormalMatrix;

out vec3 vWorldPos;
out vec3 vNormal;

void main() {
    vWorldPos = (uModel * vec4(aPos, 1.0)).xyz;
    vNormal = uNormalMatrix * aNormal; // Module 02
    gl_Position = uMVP * vec4(aPos, 1.0); // Module 03
}

// ---- fragment ----
#version 450 core
in vec3 vWorldPos, vNormal;
out vec4 fragColor;
uniform vec3 uLightPos, uLightColor, uAlbedo;

void main() {
    vec3 N = normalize(vNormal); // Module 02: renormalize
    vec3 Lv = uLightPos - vWorldPos;
    float d = length(Lv);
    vec3 L = Lv / d;
    vec3 c = uAlbedo / 3.14159 // Module 06: the 1/pi
        * uLightColor
        * max(dot(N, L), 0.0) // Module 06: cosine law
        / (d * d); // Module 06: inverse square
    fragColor = vec4(c, 1.0);
}

```

2.1 Interpolation qualifiers

Qualifier	Behaviour	Use for
smooth (default)	Perspective-correct interpolation	Almost everything: normals, UVs, world position.
noperspective	Screen-space linear	Quantities already in screen space — rare, but exactly right when you need it.
flat	No interpolation; takes the provoking vertex's value	Integer IDs, material indices, face normals. Interpolating an integer index is meaningless, and flat is how you say so.

3 How the hardware executes shaders

3.1 Lockstep execution

Shader threads are grouped into warps (NVIDIA, 32 threads) or waves (AMD, 32 or 64). Every thread in a group executes the same instruction at the same time on different data. This is what makes GPUs cheap per unit of arithmetic: one instruction decoder drives thirty-two lanes.

The consequence is **divergence**. If threads within a group would take different branches, the hardware executes both branches, masking off the lanes that are not taking each one. The cost is therefore the *sum* of the branches, not the average, and the masked lanes do no useful work.

THE PRACTICAL RULE

Before writing a branch in a fragment shader, ask: *would all 32 neighbouring pixels make the same decision?*

If yes — branching on a uniform, on a per-draw constant, on something spatially coherent — the branch is nearly free and may save a great deal.

If no — branching on a noise texture, on a per-pixel random value, on a fine checkerboard — you pay for every path and gain nothing. Prefer arithmetic selection (mix, step) in that case.

3.2 Quads and derivatives

Fragments are always shaded in 2×2 quads, because screen-space derivatives are computed as differences between neighbouring threads in the quad. $dFdx(v)$ is literally the difference between this thread's v and its horizontal neighbour's. Mipmap level selection (Module 08) depends entirely on this mechanism, which is why texture sampling inside non-uniform control flow is undefined.

Two important consequences follow. First, at a triangle edge where only one fragment of a quad is covered, the other three still execute as **helper lanes** and their results are thrown away. A triangle covering a single pixel costs four fragment-shader invocations — a $4\times$ waste. This is why very dense meshes become disproportionately expensive, and why micropolygon rendering needs a different architecture. Second, discard cannot free its lane, because the neighbours still need it for their derivatives.

4 Getting data to the GPU

Choose the mechanism by how often the data changes. Using a slower path than necessary is a common and avoidable cost; using a faster path than the data allows produces stale results.

Mechanism	Update frequency	Typical contents	Notes
Vertex attribute	Per vertex	Position, normal, UV, tangent, colour	Fixed layout declared at pipeline creation.
Uniform	Per draw call	MVP, light parameters, material scalars	Cheapest to read. Unused uniforms are optimised out, and then <code>glGetUniformLocation</code> returns <code>-1</code> — a frequent source of confusion.
Uniform buffer (UBO)	Per draw or per frame	Grouped constants: camera block, light block	Beware std140 layout rules; a <code>vec3</code> occupies 16 bytes.
Storage buffer (SSBO)	Any time, read/write	Large arrays, per-instance transforms, compute results	Unbounded size, but slower access than uniforms.
Texture	Per draw	Images, lookup tables, data maps	Hardware filtering and caching are free. Often the fastest way to read a large table.

5 What actually costs time

Graphics performance intuition is unreliable, and arithmetic is almost never the answer. In rough order of how often each one turns out to be the real bottleneck:

- 1 Draw calls and state changes.** CPU-side overhead per call is substantial. Thousands of small draws will bottleneck the CPU while the GPU idles.
- 2 Overdraw.** Fragments shaded and then covered. Sort opaque front-to-back; consider a depth pre-pass when fragment shaders are expensive.
- 3 Bandwidth.** Texture reads and render-target writes. Usually dominant at high resolution, and the reason for compressed textures and smaller render targets.
- 4 Occupancy.** A shader using too many registers reduces how many threads can be resident, which reduces the GPU's ability to hide memory latency. A small shader edit can push you over a threshold and cause a sudden frame-time cliff.
- 5 Divergence.** Idle lanes in divergent branches.
- 6 Arithmetic.** Last, and usually by a wide margin.

MEASURE FIRST

Every item above is observable. RenderDoc gives you draw counts, state, and per-draw timings; vendor tools (Nsight, Radeon GPU Profiler) give you occupancy and bandwidth.

Optimising a shader's instruction count when the frame is bandwidth-bound is a way to spend a day and gain nothing. This happens constantly.

6 Reading a frame in RenderDoc

Frame capture is not a tool of last resort; it is the normal way to find out what the GPU did. Learn it now, because every remaining module benefits.

- **Event browser** — every draw call in submission order. Check the count, and look for geometry drawn more than once.
- **Texture viewer** — inspect any render target at any point in the frame. Essential for shadow maps (Module 10) and any multi-pass effect.
- **Pipeline state** — what was actually bound when the draw executed. The majority of apparent shader bugs are binding bugs, and this view shows them in seconds.
- **Mesh viewer** — vertex buffer contents before and after the vertex shader. If the screen is black, look here first: nine times out of ten the vertices are not where you believe.
- **Shader debugger** — pick a pixel and step through its fragment shader, watching every intermediate value.

Symptom	Usual cause	Where to look
Black screen, mesh viewer output looks right	Shader failed to compile and you did not check the log.	Compile log; pipeline state.
Everything renders one flat colour	Uniform location is -1 — name typo, or the uniform was optimised out because it is unused.	Pipeline state.
Correct on your machine, wrong on another	Undefined behaviour: uninitialised variable, or depending on a driver-specific default.	Shader source; validation layers.
Correct far away, wrong close up	Wrong interpolation qualifier, or a precision issue.	Shader source.
Dark patches toward the middle of large triangles	Interpolated normals not renormalized (Module 02).	Fragment shader.
Frame time collapsed after a trivial edit	Early-Z disabled by a new discard, or register pressure reduced occupancy.	Vendor profiler.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Cem Yuksel — Interactive Computer Graphics (Utah), GPU pipeline lectures	The best free treatment of the programmable pipeline and the API layer. Primary source from this module onward.
LearnOpenGL — Getting Started and Lighting sections	Mechanics: context creation, buffers, shader compilation, uniforms. Use for the API, not for the theory.
Fabian Giesen — 'A trip through the Graphics Pipeline'	What the hardware is actually doing at each stage. The best free writing on the subject.
RenderDoc documentation and quick-start	Work through this once with a real capture. It pays for itself within the week.
The Book of Shaders	Fragment-shader intuition through small interactive examples. Good for building fluency in the language itself.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Port your Project 1 scene to OpenGL: same mesh, same camera, same lighting. Render both and diff the images. Explain every remaining difference — there will be some, and each one is informative.
- 2 Write a shader with a branch on a uniform and the same shader with the branch on a high-frequency noise texture. Measure both. Report the frame-time difference and explain it in terms of divergence.
- 3 Visualise the quad overshading: render a mesh with triangles around one pixel in size and use $dFdx/dFdy$ of a screen-space value to shade helper lanes differently. Observe the waste directly.
- 4 Capture a frame of your own renderer in RenderDoc. Write a short annotation of what each draw call contributes, and find one thing you did not know your renderer was doing.
- 5 Deliberately introduce three bugs — a typo in a uniform name, a missing normalize, and an sRGB-tagged normal map — and practise diagnosing each from the image alone before confirming in RenderDoc.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Map each of the six pipeline stages onto what you wrote in Project 1, and say which became fixed-function.
- 2 What does a divergent branch cost, and why? Give an example of a branch that is nearly free.
- 3 Why are fragments shaded in 2×2 quads, and what two consequences does that have for performance?

- 4 When would you use flat interpolation, and what goes wrong if you use smooth instead?
- 5 List four things more likely than arithmetic to be your frame's bottleneck.
- 6 `glGetUniformLocation` returns `-1` for a uniform you can see in the shader source. Give two possible explanations.